BCSE309L - Cryptography and Network Security
LAB 7: SHA512 and DSS

Name: Aryan Sharma
Register Number: 23BCE1320

SHA512 SERVER CODE

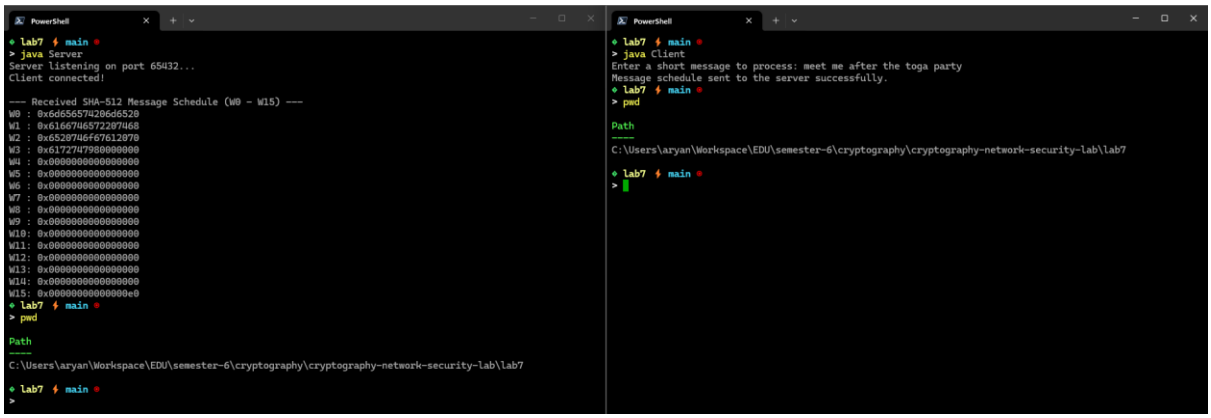
```
import java.io.ObjectInputStream;
import java.net.ServerSocket;
import java.net.Socket;

public class SHA512Server {
    public static void main(String[] args) {
        int port = 65432;
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("Server listening on port " + port + "...");

            try (Socket socket = serverSocket.accept();
                 ObjectInputStream ois = new ObjectInputStream(socket.getInputStream())) {

                System.out.println("Client connected!");
                long[] words = (long[]) ois.readObject();

                System.out.println("\n--- Received SHA-512 Message Schedule (W0 - W15) ---");
                for (int i = 0; i < words.length; i++) {
                    System.out.printf("W%-2d: 0x%016x%n", i, words[i]);
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

SHA512 CLIENT CODE

```
import java.io.ObjectOutputStream;
import java.net.Socket;
import java.nio.ByteBuffer;
import java.util.Scanner;

public class SHA512Client {

    public static long[] preprocessSHA512(String message) {
        byte[] msgBytes = message.getBytes();
        long origLenBits = (long) msgBytes.length * 8;

        ByteBuffer buffer = ByteBuffer.allocate(128);
        buffer.put(msgBytes);
        buffer.put((byte) 0x80);

        while (buffer.position() < 112) {
            buffer.put((byte) 0x00);
        }

        buffer.putLong(0L);
        buffer.putLong(origLenBits);

        buffer.flip();
        long[] wWords = new long[16];
        for (int i = 0; i < 16; i++) {
            wWords[i] = buffer.getLong();
        }

        return wWords;
    }

    public static void main(String[] args) {
        String host = "127.0.0.1";
        int port = 65432;

        try (Scanner scanner = new Scanner(System.in)) {
```

```
            System.out.print("Enter a short message to process: ");
            String message = scanner.nextLine();

            long[] words = preprocessSHA512(message);

            try (Socket socket = new Socket(host, port);
                 ObjectOutputStream oos = new ObjectOutputStream(socket.getOutputStream())) {

                oos.writeObject(words);
                System.out.println("Message schedule sent to the server successfully.");
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

SHA 512 OUTPUT



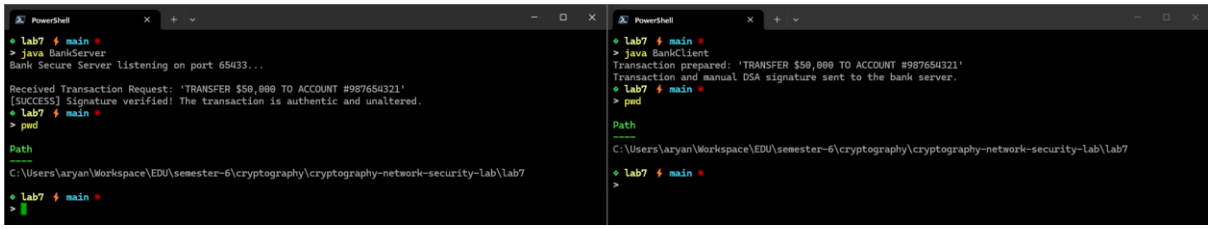
DSS SEVER CODE

```
import java.io.ObjectInputStream;
import java.math.BigInteger;
import java.net.ServerSocket;
import java.net.Socket;

public class BankServer {

    public static BigInteger getHash(String message) {
        byte[] data = message.getBytes();
        int[] h = { 0x67452301, 0xEFCDAB89, 0x98BADCFE, 0x10325476, 0xC3D2E1F0 };
        for (int i = 0; i < data.length; i++) {
            h[i % 5] = Integer.rotateLeft(h[i % 5] ^ data[i], 5);
            h[(i + 1) % 5] += h[i % 5] * 31;
        }
        byte[] out = new byte[20];
        for (int i = 0; i < 5; i++) {
            out[i * 4] = (byte) (h[i] >>> 24);
            out[i * 4 + 1] = (byte) (h[i] >>> 16);
            out[i * 4 + 2] = (byte) (h[i] >>> 8);
            out[i * 4 + 3] = (byte) h[i];
        }
        return new BigInteger(1, out);
    }
```

```
    public static void main(String[] args) {
        int port = 65433;
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("Bank Secure Server listening on port " + port + "...");
            try (Socket socket = serverSocket.accept();
                 ObjectInputStream ois = new ObjectInputStream(socket.getInputStream())) {

                BigInteger p = (BigInteger) ois.readObject();
                BigInteger q = (BigInteger) ois.readObject();
                BigInteger g = (BigInteger) ois.readObject();
                BigInteger y = (BigInteger) ois.readObject();

                String message = (String) ois.readObject();
                BigInteger r = (BigInteger) ois.readObject();
                BigInteger s = (BigInteger) ois.readObject();

                System.out.println("\nReceived Transaction Request: '" + message + "'");

                BigInteger hm = getHash(message);

                BigInteger w = s.modInverse(q);
                BigInteger u1 = hm.multiply(w).mod(q);
                BigInteger u2 = r.multiply(w).mod(q);

                BigInteger v = g.modPow(u1, p).multiply(y.modPow(u2, p)).mod(p).mod(q);

                if (v.equals(r)) {
                    System.out.println("[SUCCESS] Signature verified! The transaction is
authentic and unaltered.");
                } else {
                    System.out.println("[ERROR] Signature verification failed! Transaction
rejected.");
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

DSS CLIENT CODE

```
import java.io.ObjectOutputStream;
import java.math.BigInteger;
import java.net.Socket;
import java.util.Random;

public class BankClient {

    public static BigInteger getHash(String message) {
        byte[] data = message.getBytes();
        int[] h = { 0x67452301, 0xEFCDAB89, 0x98BADCFE, 0x10325476, 0xC3D2E1F0 };
        for (int i = 0; i < data.length; i++) {
            h[i % 5] = Integer.rotateLeft(h[i % 5] ^ data[i], 5);
            h[(i + 1) % 5] += h[i % 5] * 31;
        }
        byte[] out = new byte[20];
        for (int i = 0; i < 5; i++) {
            out[i * 4] = (byte) (h[i] >>> 24);
            out[i * 4 + 1] = (byte) (h[i] >>> 16);
            out[i * 4 + 2] = (byte) (h[i] >>> 8);
            out[i * 4 + 3] = (byte) h[i];
        }
        return new BigInteger(1, out);
    }

    public static void main(String[] args) {
        String host = "127.0.0.1";
        int port = 65433;

        try {
            Random random = new Random();

            BigInteger q = BigInteger.probablePrime(160, random);
            BigInteger p;
            BigInteger pMinus1;
            do {
                BigInteger k = new BigInteger(512 - 160, random);
```

```
            pMinus1 = k.multiply(q);
            p = pMinus1.add(BigInteger.ONE);
        } while (!p.isProbablePrime(50));

        BigInteger hNum = new BigInteger(512 - 2, random).add(BigInteger.TWO);
        BigInteger g = hNum.modPow(pMinus1.divide(q), p);

        BigInteger x;
        do {
            x = new BigInteger(160, random);
        } while (x.compareTo(BigInteger.ZERO) <= 0 || x.compareTo(q) >= 0);

        BigInteger y = g.modPow(x, p);

        String transactionMsg = "TRANSFER $50,000 TO ACCOUNT #987654321";
        System.out.println("Transaction prepared: '" + transactionMsg + "'");

        BigInteger hm = getHash(transactionMsg);

        BigInteger k;
        do {
            k = new BigInteger(160, random);
        } while (k.compareTo(BigInteger.ZERO) <= 0 || k.compareTo(q) >= 0);

        BigInteger r = g.modPow(k, p).mod(q);
        BigInteger kInv = k.modInverse(q);
        BigInteger s = kInv.multiply(hm.add(x.multiply(r))).mod(q);

        try (Socket socket = new Socket(host, port);
             ObjectOutputStream oos = new ObjectOutputStream(socket.getOutputStream())) {

            oos.writeObject(p);
            oos.writeObject(q);
            oos.writeObject(g);
            oos.writeObject(y);

            oos.writeObject(transactionMsg);
            oos.writeObject(r);
            oos.writeObject(s);

            System.out.println("Transaction and manual DSA signature sent to the bank server.");
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
  }
}
```

DSS OUTPUT